

Globale & eindeutige Zuordnung im Wissensnetz

Die drei Ebenen: Wert → Konzept · Feld → Property · Entität → Identifikator

Projekt DataBridge · 26ss_CB_DataBridge · Teilbereich Wissensnetz (Marcel Thiel)

Status: **Entwurf v0.1** · Stand: 26.08.2026

Worum es geht

Der GDC-Wrapper (Kollege A) liefert TCGA-Daten mit *lokalen* Feldnamen und Werten. Damit daraus ein **Wissensnetz** wird — und nicht nur eine weitere Datenbank — muss jedes Feld, jeder Wert und jede Entität auf eine **global gültige, eindeutige Zuordnung** abgebildet werden. Das geschieht auf drei Ebenen, die dieses Dokument erklärt.

Diese Spezifikation ist zugleich der **Übergabe-Vertrag** mit dem Team: Sie legt fest, welche Felder und Identifikatoren der Wrapper (A) liefern muss und wie der Mediator (B) sie nach RDF/OWL überführt. Solange der Wrapper gebaut wird, ist jetzt der richtige Zeitpunkt, diese Liste gemeinsam festzulegen.

Kurzüberblick der drei Ebenen:

- **Ebene 1 — Wert → globales Konzept:** ein Textwert (z. B. eine Diagnose) wird auf ein standardisiertes Ontologie-Konzept abgebildet (NCIt, GO ...).
- **Ebene 2 — Feld → Ontologie-Property/Klasse:** ein GDC-Feld/-Node wird auf eine db:-Property bzw. -Klasse abgebildet (Semantic-ETL-Regeln).
- **Ebene 3 — Entität → global eindeutiger Identifikator:** jede Entität erhält eine stabile, eindeutige IRI aus einem globalen ID-System (TCGA-Barcode, Ensembl, miRBase ...).

Ebene 1 — Wert → globales Konzept (Alignment)

Problem: Ein GDC-Feld wie `primary_diagnosis` kommt als Text, z. B. „Infiltrating duct carcinoma, NOS“. Das ist eine bloße Zeichenkette — der Graph „weiß“ nicht, welche Krankheit gemeint ist, kann sie nicht mit verwandten Diagnosen verknüpfen und keine Hierarchie abfragen.

Lösung: Eine **Alignment-Tabelle** bildet den Text auf einen **global auflösbaren Ontologie-Bezeichner (IRI)** ab. Dann ergänzt das Mapping zusätzlich zum Literal eine Aussage auf das Konzept.

Mechanismus: JSON-Tabellen unter `wissensnetz/ontology/alignment/`, gelesen von `mediator/app/semantic/mapping.py`. Nicht gelistete Werte bleiben als Literal (kein falsches Alignment).

Beispiel:

```
// wissensnetz/ontology/alignment/ncit_primary_diagnosis.json
"Infiltrating duct carcinoma, NOS": "http://purl.obolibrary.org/obo/NCIT_C4194"
```

```
# Ergebnis im Graphen (zusätzlich zum Roh-Literal):
db:diag-... db:primaryDiagnosisLabel "Infiltrating duct carcinoma, NOS" ;
           db:primaryDiagnosis    ncit:C4194 .  # globales NCIt-Konzept
```

Welche Ontologie je Werttyp? Für den Anfang genügt `primary_diagnosis` → NCIt; weitere folgen mit der TBox-Erweiterung:

Feld / Werttyp	Ziel-Ontologie	Beispiel	Status
<code>primary_diagnosis</code>	NCI Thesaurus (NCIt) / ICD-O-3	NCIT_C4194	begonnen
Krankheit / Krebsart	Disease Ontology (DOID) / NCIt	DOID_1612 (Breast Cancer)	geplant

Feld / Werttyp	Ziel-Ontologie	Beispiel	Status
Gewebe / Lokalisation	Uberon / ICD-O-Topographie	UBERON_0000310	geplant
Genfunktion	Gene Ontology (GO)	GO_0006281	geplant
Varianten-/Mutationstyp	Sequence Ontology (SO)	SO_0001583	geplant

Ebene 2 — Feld → Ontologie-Property/Klasse (Schema-Mapping)

Jedes GDC-Konstrukt bekommt ein festes Gegenstück in der DataBridge-Ontologie (Namespace db: = <http://databridge.hka/onto#>). Das sind die Semantic-ETL-Regeln aus dem Mapping-Konzept:

- **GDC-Node (Entität)** → owl:Class
- **Property + Typ** → owl:DatatypeProperty (xsd-Range)
- **Link (Beziehung)** → owl:ObjectProperty, backref → owl:inverseOf

Aktueller Stand (Kern-Ausschnitt case/project/demographic/diagnosis):

GDC-Feld / -Konstrukt	db:-Gegenstück	Typ
node: case	db:Case	owl:Class
node: project	db:Project	owl:Class
node: demographic	db:Demographic	owl:Class
node: diagnosis	db:Diagnosis	owl:Class
case.submitter_id	db:submitterId	DatatypeProperty
project.project_id	db:projectId	DatatypeProperty
demographic.gender	db:gender	DatatypeProperty
diagnoses.primary_diagnosis	db:primaryDiagnosisLabel	DatatypeProperty
diagnoses.age_at_diagnosis	db:ageAtDiagnosis	DatatypeProperty
link case→project	db:belongsToProject / db:hasCase	ObjectProperty
link diagnosis→case	db:describesCase / db:hasDiagnosis	ObjectProperty

Diese Tabelle ist der konkrete Feld-Vertrag mit dem Wrapper: Genau diese Felder müssen geliefert werden, damit das Mapping greift.

Ebene 3 — Entität → global eindeutiger Identifikator (Identität)

Damit „dasselbe Ding = derselbe Knoten“ gilt, braucht jede Entität eine **stabile, eindeutige IRI**, die aus einem **eindeutigen Schlüssel** gebildet wird. Für biologische Entitäten nutzt man dafür etablierte, weltweit eindeutige ID-Systeme.

Warum das zählt: (1) **Idempotenz** — derselbe Schlüssel erzeugt dieselbe IRI, mehrfaches Laden dupliziert nichts (RDF-Mengensemantik). (2) **Cross-Dataset-Verknüpfung** — zwei Datensätze, die dieselbe Ensembl-ID nennen, verbinden sich automatisch. Das ist die „Eindeutigkeit“ im engeren Sinn.

Entität	Eindeutiger Schlüssel	Globales ID-System	IRI-Muster
Case (Fall)	case_id / submitter_id	TCGA / GDC	.../instance/case/<id>
Project (Kohorte)	project_id	GDC	.../instance/project/<id>
Sample / Probe	aliquot/sample barcode	TCGA	.../instance/sample/<id>
Gen	Ensembl-ID (ENSG...)	Ensembl / HGNC	ensembl:ENSG...
miRNA	miRBase (MIMAT...)	miRBase	mirbase:MIMAT...

Entität	Eindeutiger Schlüssel	Globales ID-System	IRI-Muster
Protein	UniProt-Accession	UniProt	uniprot:...

Aktuell umgesetzt: Case/Project/Demographic/Diagnosis (Schlüssel aus GDC-IDs).

Gen/miRNA/Protein/Sample kommen mit der TBox-Erweiterung — und genau hier ist die Absprache mit dem Wrapper entscheidend, damit er diese IDs mitliefert.

Zusammenspiel der drei Ebenen (durchgerechnet)

Ein einzelner Fall zeigt alle drei Ebenen gleichzeitig:

```
db:case-TCGA-A1-A0SB a db:Case ;           # E3: IRI aus submitter_id | E2: Klasse
db:submitterId "TCGA-A1-A0SB" ;           # E2: Datatype-Property
db:belongsToProject db:project-TCGA-BRCA ; # E2: Object-Property (Relation)
db:hasDiagnosis db:diag-d-11111111 .

db:diag-d-11111111 a db:Diagnosis ;
db:primaryDiagnosisLabel "Infiltrating duct carcinoma, NOS" ; # Rohwert
db:primaryDiagnosis ncit:C4194 .           # E1: Alignment auf globales NCIt-Konzept
```

E3 sorgt dafür, dass dieser Fall überall derselbe Knoten ist; E2 gibt ihm Struktur; E1 verbindet seine Diagnose mit der weltweiten Onkologie-Terminologie.

Was der Wrapper dafür liefern muss

Damit die drei Ebenen greifen, muss der GDC-Wrapper je Fall mindestens folgende Felder ausgeben (Kern-Ausschnitt, erweiterbar):

- **Identität (E3):** case_id und submitter_id, diagnoses[].diagnosis_id, project.project_id.
- **Attribute (E2):** demographic.gender, diagnoses[].age_at_diagnosis.
- **Alignment-Werte (E1):** diagnoses[].primary_diagnosis (Text exakt wie von GDC).
- **Für spätere Ausbaustufen:** Genexpression mit Ensembl-IDs, miRNA mit miRBase-Accessions, Sample-/Aliquot-Barcodes — damit E3 auch für die biologischen Entitäten trägt.

Offene Punkte & nächste Schritte

- **E1:** die vorkommenden primary_diagnosis-Werte ermitteln (SPARQL) und ihre verifizierten NCIt-Codes eintragen.
- **E2/E3:** TBox um Sample/Genexpression/miRNA/Methylierung erweitern und die zugehörigen ID-Systeme (Ensembl/miRBase/UniProt) festlegen.
- **Team:** diese Tabellen mit Wrapper (A) und Mediator (B) abstimmen — sie sind der gemeinsame Übergabe-Vertrag.

Referenzen (globale ID-/Ontologie-Systeme)

- [NCI Thesaurus \(NCIt\) — EBI OLS](#)
- [Disease Ontology \(DOID\)](#)
- [Gene Ontology \(GO\)](#)
- [Sequence Ontology \(SO\)](#)
- [Uberon \(Anatomie\)](#)
- [Ensembl \(Gen-IDs\)](#)
- [miRBase \(miRNA\)](#)

- [UniProt \(Proteine\)](#)